

```
5 59 secondary 0 0.00
6 35 tertiary 747 806.76
...
```

pull

The *pull()* function accepts a numeric argument for the column number and returns its values. The last column in the data set is indexed at '-1' and corresponds to whether the client had subscribed to a term deposit.

```
> bank %>% pull(-1)
[1] "no" "no" "no" "no" "no" "no" "no" "no" "no"
"no" "no" "no"
[13] "no" "yes" "no" "no" "no" "no" "no" "no" "no"
"no" "no" "no"
[25] "no" "no" "no" "no" "no" "no" "yes" "no" "no"
"yes" "yes" "no"
[37] "yes" "yes" "yes" "no" "no" "no" "no" "no" "no"
"no" "no" "no"
```

group_by

The data set can be categorised using the *group_by()* function, as demonstrated below:

```
> bank %>% group_by(age)
# A tibble: 4,521 × 17
# Groups:   age [67]
  age job marital education default balance housing
  <int> <chr> <chr> <chr> <chr> <int> <chr>
<chr> <chr> <int>
1 30 unemployed married primary no 1787 no
no cellul... 19
2 33 services married secondary no 4789 yes
yes cellul... 11
3 35 managem... single tertiary no 1350 yes
no cellul... 16
4 30 managem... married tertiary no 1476 yes
yes unknown 3
5 59 blue-col... married secondary 0 yes
no unknown 5
6 35 managem... single tertiary no 747 no
no cellul... 23
7 36 self-emp... married tertiary no 307 yes
no cellul... 14
8 39 technici... married secondary no 147 yes
no cellul... 6
9 41 entrepre... married tertiary no 221 yes
no unknown 14
10 43 services married primary no -88 yes
yes cellul... 17
# i 4,511 more rows
...
```

summarise

You can create a new data frame by also grouping variables using the *summarise()* function. The *summarize()* name can also be used instead.

```
> bank %>% summarise(age, balance)
  age balance
1 30 1787
2 33 4789
3 35 1350
4 30 1476
5 59 0
```

glimpse

The entire data frame can be transposed using the *glimpse()* function, which shows every column as a row. It tries to show as much of the data as possible in the output.

```
> bank %>% glimpse()
Rows: 4,521
Columns: 17
$ age <int> 30, 33, 35, 30, 59, 35, 36, 39, 41, 43, 39,
43, 36, 20, 31, ...
$ job <chr> "unemployed", "services", "management",
"management", "blue-...
$ marital <chr> "married", "married", "single", "married",
"married", "singl...
$ education <chr> "primary", "secondary", "tertiary",
"tertiary", "secondary",...
$ default <chr> "no", "no", "no", "no", "no", "no", "no",
"no", "no", "no", ...
$ balance <int> 1787, 4789, 1350, 1476, 0, 747, 307, 147,
221, -88, 9374, 26...
```

slice

You can select the first three rows by specifying them with the *slice()* function. It allows you to select, filter and show duplicate rows as well. For example:

```
> bank %>% slice(1:3)
  age job marital education default balance housing
loan contact day
1 30 unemployed married primary no 1787 no
no cellular 19
2 33 services married secondary no 4789 yes
yes cellular 11
3 35 management single tertiary no 1350 yes
no cellular 16
month duration campaign pdays previous poutcome y
1 oct 79 1 -1 0 unknown no
2 may 220 1 339 4 failure no
3 apr 185 1 330 1 failure no
```

Continued to page...78